

Inteligência Computacional - Algoritmos Genéticos

O problema das N-rainhas

Autores:

Arthur Faria

Gustavo D. Ventino

Universidade Federal de Uberlândia

Bacharelado em Ciências da Computação - Inteligência Computacional

Fevereiro de 2025

SUMÁRIO

Sumário	1	
0.1	Conceitos Básicos	2
0.1.1	Resumo	2
0.1.2	O que são algoritmos genéticos?	2
0.1.3	Aspectos Fundamentais	3
0.2	O problema das N-rainhas	5
0.2.1	Descrição do Problema	5
0.2.2	Descrição da Solução	5
0.2.3	Representação do Indivíduo	5
0.2.4	Operadores Genéticos	6
0.2.5	Função de Adaptação	7
0.2.6	Nossa função de Adaptação	8
0.3	Parâmetros de funcionamento do algoritmo	9
0.3.1	Critério de Parada	9
0.3.2	Probabilidades dos Operadores Genéticos	9
0.3.3	Nossas probabilidades de Operadores Genéticos	9
0.3.4	Benchmarks	10
0.3.5	Benchmarks próprios	12
0.3.6	Resultados Adicionais	13
0.4	Conclusão	14
	REFERÊNCIAS	15

0.1 Conceitos Básicos

0.1.1 RESUMO

O objetivo desta seção é apresentar de maneira resumida os principais conceitos e termos que envolvem Algoritmos Genéticos, a fim de consolidar algum tipo de conhecimento antes de tratarmos do problema objeto deste documento. Isso se dá pois acreditamos fielmente que uma recapitulação destes tópicos pode auxiliar no entendimento do problema que será apresentado.

0.1.2 O QUE SÃO ALGORITMOS GENÉTICOS?

Existem diversas definições para algoritmos genéticos, iremos abordar duas, as quais entendemos como boas definições.

De acordo com ([CARVALHO](#),), temos:

”Algoritmos Genéticos são algoritmos de otimização global, baseados nos mecanismos de seleção natural e da genética. Eles empregam uma estratégia de busca paralela e estruturada, mas aleatória, que é voltada em direção ao reforço da busca de pontos de "alta aptidão", ou seja, pontos nos quais a função a ser minimizada (ou maximizada) tem valores relativamente baixos (ou altos). Apesar de aleatórios, eles não são caminhadas aleatórias não direcionadas, pois exploram informações históricas para encontrar novos pontos de busca onde são esperados melhores desempenhos. Isto é feito através de processos iterativos, onde cada iteração é chamada de geração.”

De acordo com ([PACHECO](#),), temos:

”Algoritmos Genéticos (Gas-Genetic Algorithms) constituem uma técnica de busca e otimização, altamente paralela, inspirada no princípio Darwiniano de seleção natural e reprodução genética. Os princípios da natureza nos quais os GAs se inspiram são simples. De acordo com a teoria de C. Darwin, o princípio de seleção privilegia os indivíduos mais aptos com maior longevidade e, portanto, com maior probabilidade de reprodução. Indivíduos com mais descendentes têm mais chance de perpetuarem seus códigos genéticos nas próximas gerações. Tais códigos genéticos constituem a identidade de cada indivíduo e estão representados nos cromossomos. Estes princípios são imitados na construção de algoritmos computacionais que buscam uma melhor solução para um determinado problema, através da evolução de populações de soluções codificadas através de cromossomos artificiais”

Entende-se portanto, que temos uma classe de algoritmos voltados em encontrar uma solução ótima, ou uma solução que nos satisfaça. Para isso, reproduzimos princípios naturais de evolução de uma espécie. Mas para que isso seja possível, é necessário entendermos quais são aspectos fundamentais dos nossos algoritmos genéticos.

Vejamos a seguir uma explicação genérica do funcionamento de um algoritmo genérico:

1. Uma população aleatória de soluções candidatas é criada e os valores de *fitness* dos indivíduos são calculados. Em seguida, os cromossomos na população são ordenados e classificados de acordo com seus valores de *fitness*.
2. Um certo número de cromossomos será passado para a próxima geração com base em um operador de seleção, favorecendo os indivíduos mais aptos conforme sua classificação na população.
3. Os cromossomos selecionados atuam como pais e participam da operação de cruzamento para gerar descendentes, cujos valores de *fitness* são calculados simultaneamente. A probabilidade de cruzamento geralmente é mantida alta, pois tende a produzir descendentes mais aptos.
4. Com base em uma probabilidade de mutação, um operador de mutação é aplicado aos novos indivíduos, modificando aleatoriamente alguns cromossomos. A probabilidade de mutação geralmente é mantida baixa.
5. Os descendentes avaliados, juntamente com seus pais, formam a população para a próxima geração.

0.1.3 ASPECTOS FUNDAMENTAIS

Um algoritmo genético pode ser formalmente representado como uma tupla $(P, f, \Omega, s, r, m, t)$, onde cada componente representa um aspecto fundamental do processo evolutivo:

- **P (População):** Conjunto finito de indivíduos $P = \{p_1, p_2, \dots, p_n\}$, onde cada indivíduo p_i é tipicamente codificado como uma sequência (cromossomo) de símbolos (genes) provenientes de um alfabeto finito. Cada cromossomo representa uma solução candidata no espaço de busca do problema.
- **f (Função de *fitness*):** Uma função $f : P \rightarrow R$ que associa a cada indivíduo $p_i \in P$ um valor real $f(p_i)$ que quantifica sua qualidade como solução. Para problemas de maximização: quanto maior o valor de $f(p_i)$, melhor a solução; para minimização: quanto menor o valor, melhor a solução.
- **Ω (Critério de seleção):** Mecanismo que define a probabilidade de seleção

$$\Omega(p_i) = \frac{f(p_i)}{\sum_{j=1}^n f(p_j)}$$

para cada indivíduo, estabelecendo uma distribuição de probabilidade sobre a população, a fim de guiar o processo de seleção baseado no *fitness* relativo. Esta função também pode variar, dependendo do problema e dos objetivos dos autores da solução.

- **s (Operador de seleção):** Função $s : P^n \rightarrow P^m$ que seleciona m indivíduos da população para reprodução, conforme as probabilidades estabelecidas por Ω . Implementações comuns incluem seleção por roleta, torneio e ranking. Existem também outros mecanismos para seleção, como *Random Picking*, Torneio, *etc.*
- **r (Operador de *crossover*):** Função $r : P^2 \rightarrow P^2$ que combina características de dois indivíduos parentais para gerar novos descendentes. Se $p_i = (g_1^i, g_2^i, \dots, g_l^i)$ e $p_j = (g_1^j, g_2^j, \dots, g_l^j)$ são os cromossomos parentais, o crossover em um ponto k produz descendentes $(g_1^i, \dots, g_k^i, g_{k+1}^j, \dots, g_l^j)$ e $(g_1^j, \dots, g_k^j, g_{k+1}^i, \dots, g_l^i)$. Pode gerar um número variável de indivíduos, nem sempre gera dois indivíduos, pode gerar mais ou menos "filhos".
- **m (Operador de mutação):** Função $m : P \rightarrow P$ que modifica aleatoriamente genes de um cromossomo com uma probabilidade p_m . Para um gene g_k em um cromossomo, a mutação pode ser representada como $m(g_k) = g'_k$, onde g'_k é um valor alternativo do alfabeto de genes.
- **t (Critério de parada):** Condição lógica $t : \mathcal{H} \rightarrow \{\text{verdadeiro, falso}\}$ que avalia o histórico $\mathcal{H}$ das populações geradas e determina se o algoritmo deve encerrar sua execução. Critérios comuns incluem: atingir um número máximo de gerações, encontrar uma solução com aptidão acima de um limiar predefinido, ou detectar estagnação na evolução da população (convergência).

O algoritmo evolui iterativamente, partindo de P_0 (população inicial) e gerando sucessivas populações $P_1, P_2, \dots, P_g$ através da aplicação sequencial dos operadores genéticos seleção, *crossover* e mutação, até que o critério de término t seja satisfeito.

0.2 O problema das N-rainhas

0.2.1 DESCRIÇÃO DO PROBLEMA

Antes de apresentarmos a descrição do problema, vamos entender a motivação de desenvolver um algoritmo genético para tal problema:

NP (tempo polinomial não determinístico) é a classe de problemas que são solucionáveis em tempo polinomial por uma máquina de Turing não determinística. Em outras palavras, problemas que não possuem soluções determinísticas que sejam executadas em tempo polinomial são chamados de problemas de classe NP. Eles não podem ser resolvidos em tempo real usando abordagens determinísticas devido à sua alta complexidade, da ordem de 2^n ou $n!$. Métodos heurísticos são utilizados para resolver esses tipos de problemas em tempo real. O Problema das N Rainhas é um problema de otimização atribuído à classe de Problemas NP-Completo. Daí vem a necessidade de desenvolver um AG para resolver tal problema, uma vez que este se é um algoritmo de busca global com heurística.

Agora sim podemos descrever o problema, O objetivo do Problema das N Rainhas é posicionar adequadamente N Rainhas em um tabuleiro de xadrez $N \times N$ de forma que não haja conflito entre elas devido ao arranjo, ou seja, não há interseção entre elas verticalmente, horizontalmente ou diagonalmente. O Problema das Oito Rainhas é apenas um caso especial do Problema das N Rainhas, onde N é igual a 8.

0.2.2 DESCRIÇÃO DA SOLUÇÃO

A solução para o problema consiste em posicionar adequadamente N Rainhas em um tabuleiro de xadrez $N \times N$ de forma que não haja conflito entre elas devido ao arranjo, ou seja, não há interseção entre elas verticalmente, horizontalmente ou diagonalmente

0.2.3 REPRESENTAÇÃO DO INDIVÍDUO

O artigo escolhido ([SARKAR; NAG, 2017](#)) representa os indivíduos por meio de uma N-tupla $(c_1, c_2, c_3 \dots c_N)$, em que c_i representa a posição da rainha na i -ésima coluna e na c -ésima linha. Seguimos com a mesma representação do artigo no nosso algoritmo. Vale ressaltar que como cada indivíduo é um tabuleiro, estes poderiam ser representados por meio de uma matriz binária que representa o tabuleiro, onde 0 é onde não temos uma rainha e 1 é onde temos uma rainha. O problema desta representação é que ela ocupa muito espaço em memória e adiciona complexidade à função de *crossover*.

0.2.4 OPERADORES GENÉTICOS

Temos os seguintes operadores genéticos:

Crossover: Como consideramos cada cromossomo como uma permutação aleatória de $(1, 2, 3 \dots, N)$, então é necessário projetar uma *crossover* que realiza a permutação. Os autores do artigo projetaram um *crossover* de Ordem 1. O Crossover de Ordem 1 é um tipo de cruzamento baseado em permutação muito simples, no qual dois pontos aleatórios são selecionados do pai 1 e os alelos entre esses pontos são transferidos diretamente para o filho. Em seguida, os outros alelos do pai 2 que ainda não estão presentes no filho são adicionados na mesma ordem em que aparecem no pai 2. Seguimos este operador de crossover à risca, embora houvessem outras opções, como gerar mais de um filho por crossover fundindo partes complementares dos pais.

Ex:

Pai 1	5	2	<u>3</u>	<u>1</u>	6	4	8	7
Pai 2	1	8	6	4	7	5	3	2
Filho	8	7	3	1	6	4	5	2

- **Passo 1:** No cenário descrito, os pontos de alelos 3 e 4 são selecionados aleatoriamente do pai 1, e os alelos entre esses dois pontos são transferidos diretamente para o cromossomo do filho.
- **Passo 2:** Os alelos equivalentes (ou iguais) aos alelos entre os pontos de cruzamento do pai 1 são eliminados do pai 2.
- **Passo 3:** Os alelos restantes do pai 2 são inseridos no cromossomo do filho na mesma ordem em que aparecem no pai 2.

Repositório de População Fraca: Embora este não seja um operador genético por definição, ele funciona como um operador de seleção para as novas gerações da população. Os autores do artigo criaram um repositório de cromossomos fracos. O tamanho do repositório é dado por $\lfloor \sqrt{N} \rfloor$, onde $\lfloor x \rfloor$ representa o maior inteiro menor ou igual a x .

1. Em cada iteração, dois cromossomos são selecionados aleatoriamente e passam por cruzamento e mutação. O descendente gerado compete com os cromossomos da população principal. Se o descendente for mais apto que os piores cromossomos da população principal, ele substitui esses cromossomos.
2. Em cada iteração, um cromossomo é selecionado aleatoriamente e cruzado com outro cromossomo aleatório da população principal. O descendente gerado também compete com os cromossomos da população principal por sua permanência.

Essa abordagem ajuda a evitar a convergência para ótimos locais. Durante toda a execução, este repositório é mantido constante, sem atualizações nos cromossomos armazenados. Não utilizamos esse repositório em nosso trabalho.

Torneio: Nós fizemos diferente dos autores do artigo, fizemos a escolha dos indivíduos para próxima geração e reprodução por meio de torneio dos indivíduos. Sorteamos 3 indivíduos e os selecionamos para competir com base no valor do *fitness*, o indivíduo com maior *fitness* que seus competidores é considerado vencedor e é atribuído à próxima geração. O processo se repete até termos uma geração com população do tamanho desejado.

Mutação: O operador de mutação é muito importante em algoritmos genéticos, uma vez que eles evitam que nosso algoritmo presas em um ótimo local. Dito isso, o artigo descreve o seguinte operador de mutação, selecionamos dois alelos aleatoriamente e trocamos eles de posição um com o outro, esta é denominada mutação simples. Podemos realizar também a mutação dupla, em que realizamos duas mutações simples, a taxa de mutação simples proposta é de 0.8, e a de mutação dupla é 0.4. Realizamos apenas mutações simples, e quanto à taxa de mutação encontramos empiricamente o valor 0.2

Antes da mutação simples: 2 6 8 3 4 7 5
 Depois da mutação simples: 2 6 7 3 4 8 5

0.2.5 FUNÇÃO DE ADAPTAÇÃO

Os autores do artigo projetaram a função de *fitness* de cada cromossomo de forma que a presença de k rainhas na mesma diagonal acumule $(k - 1)$ pontos para o valor de aptidão. Todos esses pontos são somados para obter o valor final da aptidão.

Considere um tabuleiro de xadrez $N \times N$ com a disposição das rainhas representada por $(c_1, c_2, c_3, \dots, c_N)$. Primeiramente, verificamos se mais de uma rainha está na mesma diagonal com direção ($\nearrow$). Isso ocorre se:

$$\forall i, j \in \{1, 2, \dots, N\}, \quad i \neq j, \quad (c_i - i) = (c_j - j)$$

De forma semelhante, verificamos se mais de uma rainha está na mesma diagonal com direção ($\nwarrow$). Isso ocorre se:

$$\forall i, j \in \{1, 2, \dots, N\}, \quad i \neq j, \quad (c_i + i) = (c_j + j)$$

Portanto, acumulamos o valor de aptidão sempre que qualquer uma das condições acima for satisfeita. Isso indica claramente que o cromossomo mais apto possui um valor de aptidão igual a 0.

Algorithm 1 Função de Aptidão (fitness)

```
1: function FITNESS(cromossomo)
2:    $t1 \leftarrow 0$  ▷ Número de rainhas repetidas na mesma diagonal vista do canto esquerdo
3:    $t2 \leftarrow 0$  ▷ Número de rainhas repetidas na mesma diagonal vista do canto direito
4:    $size \leftarrow$  comprimento do cromossomo
5:   Definir  $f1, f2$  como vetores de tamanho  $size$ 
6:   for  $i \leftarrow 1$  to  $size$  do
7:      $f1[i] \leftarrow (cromossomo[i] - i)$ 
8:      $f2[i] \leftarrow ((1 + size) - cromossomo[i] - i)$ 
9:   end for
10:  Ordenar  $f1$ 
11:  Ordenar  $f2$ 
12:  for  $i \leftarrow 2$  to  $size$  do
13:    if  $f1[i] = f1[i - 1]$  then ▷ Verifica se duas rainhas estão na mesma diagonal
    do canto esquerdo
14:       $t1 \leftarrow t1 + 1$ 
15:    end if
16:    if  $f2[i] = f2[i - 1]$  then ▷ Verifica se duas rainhas estão na mesma diagonal
    do canto direito
17:       $t2 \leftarrow t2 + 1$ 
18:    end if
19:  end for
20:   $fitness\_value \leftarrow t1 + t2$ 
21:  return  $fitness\_value$ 
22: end function
```

0.2.6 NOSSA FUNÇÃO DE ADAPTAÇÃO

Nossa função de *fitness* foi projetada de outra forma. Ela quantifica o conflito entre as rainhas e converte esse valor em uma pontuação normalizada entre 0 e 1, onde 1 representa uma solução perfeita.

A lógica é similar. Verificamos as diagonais para contabilizar os conflitos e a função de fitness é dada por:

$$f(x) = e^{-c(x)}$$

Onde:

$f(x)$ representa a função de fitness

$c(x)$ representa o número total de conflitos na solução x

e é o número de euler a base do logaritmo natural

0.3 Parâmetros de funcionamento do algoritmo

O algoritmo tem pouquíssimos parâmetros de funcionamento, sendo eles: o tamanho do tabuleiro e quantidade de rainhas N , e as taxas de mutação simples e dupla.

0.3.1 CRITÉRIO DE PARADA

O critério de parada utilizado foi avaliar se algum cromossomo da população tinha $fitness = 0$, isto é, se há um cromossomo que resolve o problema de fato. Como nossa função de $fitness$ é diferente, buscamos avaliar se há ao menos um cromossomo com $fitness = 1$.

0.3.2 PROBABILIDADES DOS OPERADORES GENÉTICOS

As probabilidades utilizadas no artigo foram:

$$mutação_{simples} = 0.8$$

$$mutação_{dupla} = 0.4$$

$$ponto_{crossover_1} = \frac{1}{N}$$

$$ponto_{crossover_2} = \frac{1}{N - 1}$$

As probabilidades utilizadas na nossa proposta foram:

$$mutação_{simples} = 0.2$$

$$ponto_{crossover_1} = \frac{1}{N}$$

$$ponto_{crossover_2} = \frac{1}{N - 1}$$

0.3.3 NOSSAS PROBABILIDADES DE OPERADORES GENÉTICOS

Nosso crossover segue as mesmas probabilidades do artigo porém não realizamos mutação dupla. Cada indivíduo só pode sofrer uma mutação após ser gerado pelo crossover de seus pais. Após experimentação de alguns valores chegamos a uma taxa de $mutação_{simples} = 0.2$

0.3.4 BENCHMARKS

O teste realizado pelos autores do artigo consiste em: Considerando valores de

$$N = 10, 11, \dots, 25.$$

Utilizar o algoritmo metaheurístico para encontrar três soluções ótimas para cada valor N .

Para cada execução, o número de iterações necessário para que o algoritmo atinja o ótimo global também é apresentado. As soluções obtidas, ou seja, os elementos das N -tuplas, são exibidos no formato de tabela na coluna ***Solução Ótima***.

Os resultados mostram claramente que as soluções ótimas foram encontradas com um número significativamente menor de iterações.

N	Iterações	Solução Ótima
25	1431	22 6 11 14 24 15 19 8 4 17 1 3 12 16 7 21 18 2 25 23 10 20 5 9 13
	382	12 4 16 23 11 24 22 20 2 5 15 10 7 19 3 8 17 25 6 1 18 13 9 14 21
	281	17 5 22 8 12 21 1 13 20 16 19 24 11 23 4 7 3 15 9 2 14 10 18 25 6
24	4043	6 8 15 17 14 10 23 18 24 2 4 11 7 3 16 13 19 9 22 5 21 12 1 20
	392	13 3 22 6 18 2 23 11 4 14 17 19 24 8 20 5 1 15 10 7 21 12 9 16
	420	6 14 21 17 24 16 1 7 4 10 3 18 15 23 11 19 2 5 8 13 22 20 9 12
23	1363	5 14 22 3 8 13 21 4 6 23 19 2 18 7 17 11 9 16 10 20 15 1 12
	113	18 16 21 17 3 6 13 1 4 7 5 22 14 19 11 15 8 20 9 23 2 10 12
	347	4 16 14 23 1 10 22 15 11 5 2 18 21 8 13 17 7 3 19 6 20 9 12
22	282	8 13 1 9 14 19 4 22 15 12 6 11 21 18 3 17 20 10 2 16 5 7
	2044	13 18 12 5 8 1 11 16 2 6 21 9 15 19 22 3 17 4 7 10 20 14
	1082	18 3 12 2 16 5 11 1 15 20 6 4 9 19 17 10 14 7 22 8 21 13
21	404	6 11 16 10 15 21 3 1 9 14 5 19 2 12 18 7 4 17 20 8 13
	807	7 19 13 16 12 5 3 1 6 2 11 20 14 17 4 18 21 8 10 15 9
	592	16 5 13 21 7 18 11 3 1 4 10 17 19 12 15 6 14 9 20 8 2
20	453	12 7 11 3 20 18 14 4 6 8 5 15 17 9 2 16 19 10 1 13
	279	9 17 1 10 2 16 18 8 12 3 15 6 20 13 5 7 19 14 11 4
	170	4 16 9 6 1 17 20 5 13 11 18 2 7 12 15 3 8 10 14 19
19	3166	1 5 9 13 17 8 12 15 6 3 19 4 14 18 10 2 11 16 7
	1462	2 15 9 11 13 8 1 3 19 10 14 17 6 18 12 5 16 4 7
	382	8 5 2 13 9 16 12 19 17 6 4 1 7 14 10 18 15 3 11
18	5730	12 3 11 13 5 9 1 4 7 16 10 17 15 18 8 2 14 6
	271	13 4 16 1 10 7 3 12 17 2 6 18 11 14 8 15 5 9
	1828	3 11 13 5 1 18 14 2 8 15 9 7 17 4 12 16 6 10
14	49	11 1 10 5 9 2 12 14 7 13 4 6 8 3
	1077	1 14 7 2 12 9 6 3 10 4 13 8 5 11
	31	2 6 11 9 7 1 4 14 12 8 5 3 13 10
13	2272	12 6 3 10 4 11 8 2 7 13 1 9 5
	224	9 12 8 3 1 13 5 10 6 11 2 4 7
	8	8 13 3 6 11 5 12 9 4 2 7 10 1
12	2865	6 2 5 10 12 4 8 11 3 1 7 9
	82	2 8 5 9 1 6 11 3 12 7 4 10
	52	4 9 1 3 11 8 12 2 6 10 7 5

11	320	10 8 6 3 9 11 1 5 7 2 4
	339	8 10 2 4 9 7 3 11 6 1 5
	97	3 9 6 4 11 1 8 2 5 7 10
10	2383	6 8 1 4 9 5 2 10 3 7
	125	4 10 7 2 6 3 1 8 5 9
	251	5 7 4 1 8 2 9 6 3 10

0.3.5 BENCHMARKS PRÓPRIOS

Para avaliar a eficácia do algoritmo genético proposto, foram realizados testes extensivos com diferentes tamanhos de tabuleiros. Os testes foram realizados considerando uma população inicial de 100 indivíduos, com um máximo de 5000 gerações e uma taxa de mutação de 0.1. Para cada valor de N (tamanho do tabuleiro) foram realizados 10 experimentos independentes.

Os resultados detalhados são apresentados a seguir, incluindo a taxa de sucesso (porcentagem de execuções que encontraram uma solução ótima), o número médio de gerações necessárias para convergência e o tempo médio de execução.

Tabela 2 – Resultados do algoritmo genético para o problema das N -rainhas

N	Taxa de Sucesso	Gerações Médias	Tempo Médio
5	100,0%	0,0	0,0s
6	100,0%	1,2	0,0s
7	100,0%	1,0	0,0s
8	100,0%	5,0	0,0s
9	100,0%	3,0	0,0s
10	100,0%	285,4	0,5s
11	70,0%	78,9	2,6s
12	100,0%	418,6	0,8s
13	90,0%	599,4	2,0s
14	100,0%	528,8	1,1s
15	80,0%	624,2	3,2s
16	60,0%	819,5	5,8s
17	70,0%	189,0	4,1s
18	80,0%	629,1	4,1s
19	80,0%	1583,8	6,6s
20	80,0%	949,9	5,5s
21	60,0%	261,7	7,5s
22	80,0%	1140,9	6,7s
23	90,0%	1544,6	7,0s
24	90,0%	547,6	4,1s
25	100,0%	817,0	3,6s

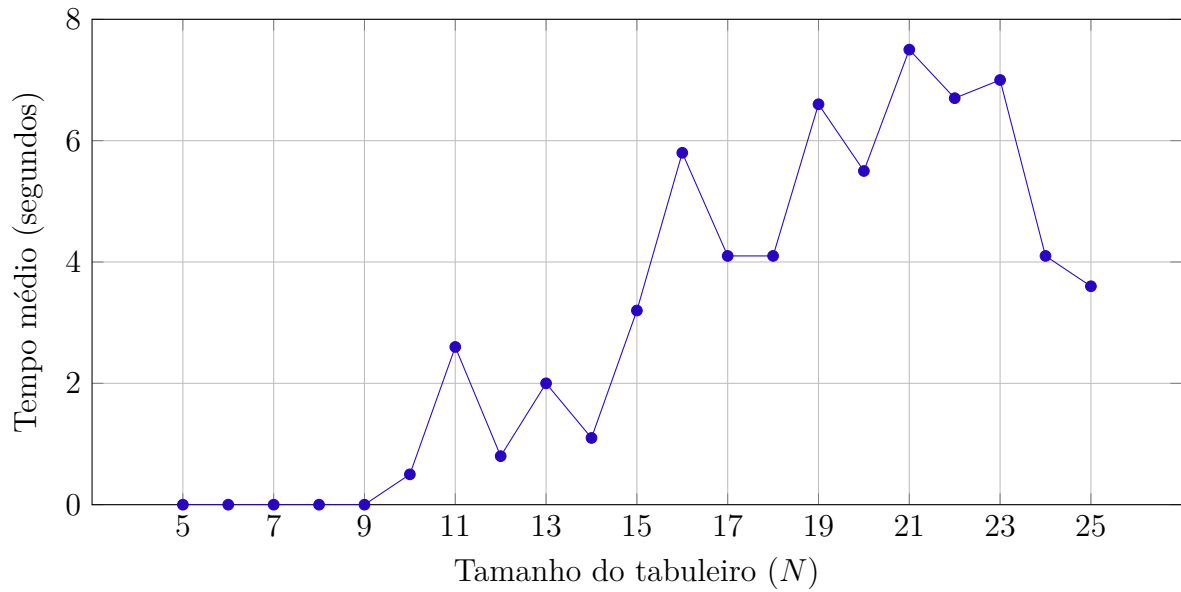


Figura 1 – Tempo médio de execução para diferentes valores de N

0.3.6 RESULTADOS ADICIONAIS

Em uma série adicional de experimentos, estendemos a análise para incluir tabuleiros de tamanho até $N = 30$. A Tabela 3 apresenta estes resultados complementares.

Tabela 3 – Resultados adicionais do algoritmo genético para o problema das N -rainhas

N	Taxa de Sucesso	Gerações Médias	Tempo Médio
5	100,0%	0,0	0,0s
6	100,0%	0,8	0,0s
7	100,0%	0,4	0,0s
8	100,0%	6,5	0,0s
9	100,0%	2,7	0,0s
10	100,0%	585,4	0,9s
15	90,0%	634,2	2,3s
20	90,0%	960,1	4,1s
25	90,0%	348,0	3,3s
30	100,0%	506,2	2,7s

Como podemos perceber não há muita constância entre as execuções do AG. Frequentemente conseguimos resultados mais rápidos para instâncias mais difíceis. Resultado comum, afinal estamos trabalhando com métodos com muita aleatoriedade. Mais experimentos podem ser encontrados no repositório.

0.4 Conclusão

O artigo revela que o problema das N-Rainhas pode ser resolvido em um tempo razoável utilizando meta-heurísticas, como o Algoritmo Genético e suas versões modificadas. O problema das N-Rainhas tem uma aplicação prática limitada, mas representa uma classe importante de problemas NP, que não podem ser resolvidos em um tempo viável por métodos determinísticos.

Os Algoritmos Genéticos foram concebidos como heurísticas para resolver problemas com soluções boas e ruins, dependendo dos valores de aptidão das soluções. No entanto, neste problema eles demonstraram sua capacidade de resolver problemas de otimização combinatória com respostas binárias simples (sim ou não).

Houveram trabalhos anteriores que tentaram resolver o problema das N-Rainhas utilizando Algoritmos Genéticos. Este artigo, realizou algumas modificações nas funções de aptidão, o que resultou em uma nova versão modificada do Algoritmo Genético, essa abordagem proporcionou resultados promissores e satisfatórios em menos tempo, quando comparada às abordagens anteriores.

REFERÊNCIAS

CARVALHO, A. C. P. de Leon Ferreira de. *Genetic Algorithms*. <<https://sites.icmc.usp.br/andre/research/genetic/>>. Acessado em: 02 de abril de 2025. Citado na página 2.

PACHECO, M. A. C. *Algoritmos Genéticos: Princípios e Aplicações*. Rua Marques de São Vicente 225, Gávea, CEP 22453-900, Rio de Janeiro, RJ, Brasil. Email: marco@ele.puc-rio.br, Tel: (+55 21) 2529-9445. Citado na página 2.

SARKAR, U.; NAG, S. *An Adaptive Genetic Algorithm for Solving N-Queens Problem*. 2017. Disponível em: <<https://arxiv.org/abs/1802.02006>>. Citado na página 5.